

Proyecto de Fin de Ciclo

2º Desarrollo de Aplicaciones Web



Proyecto · Estadísticas inteligentes de Restaurante

Sistema de Dashboard de Ventas para Restaurante con Arquitectura Serverless en AWS

Autor: Muhammad Salman Bajwa

Curso: 2º DAW · **Año:** 2025/2026

Centro: IES LA VEREDA

Tutor/a: Belén Tejera Maica

Fecha: Mayo 2026

Resumen del contenido (contraportada)

Este documento describe el diseño, implementación y despliegue de un **dashboard de ventas para restaurante** (*Restaurante Estrella*) basado en **arquitectura serverless en AWS**. El frontend es un sitio estático (HTML5, CSS3, JavaScript ES6 modules) alojado en **Amazon S3**; el backend es una función **AWS Lambda con Node.js 24.x** expuesta mediante **API Gateway**; los datos persisten en **Amazon RDS MySQL 8.0**. Incluye manual de usuario, manual técnico, modelo de datos (DDL/DML/DCL/TCL), integración API–frontend con Chart.js, y análisis de costes y conclusiones.

RESUMEN

Este proyecto presenta el desarrollo de un sistema completo de dashboard de ventas para restaurante utilizando arquitectura serverless en Amazon Web Services (AWS). La aplicación permite la visualización de datos de ventas agregados mediante gráficos interactivos, implementando **AWS Lambda (Node.js 24.x)**, **API Gateway**, **RDS MySQL** y **S3** como hosting web estático.

El sistema combina un frontend responsive (Bootstrap 5, Chart.js, soporte bilingüe español/inglés y modo oscuro) con un backend serverless que expone una API REST (GET /sales) y agrega transacciones de la base relacional para devolver el array salesRecords[] consumido por el panel. La arquitectura prioriza escalabilidad automática, alta disponibilidad gestionada por AWS y coste optimizado por uso.

La elección de **RDS MySQL** responde a requisitos académicos de base de datos relacional (modelo entidad-relación, SQL DDL/DML/DCL/TCL). Se reconoce que **DynamoDB** sería más idónea en un entorno serverless puro.

Palabras clave: AWS, Serverless, Dashboard, Ventas, Lambda, Node.js, API Gateway, RDS, S3, Chart.js

Primera página interior

Sistema de Dashboard de Ventas para Restaurante con Arquitectura Serverless en AWS

(Muhammad Salman Bajwa)

ÍNDICE DE CONTENIDOS

1. Introducción
 - 1.1. Contexto y resumen del documento
 - 1.2. Objetivos
 - 1.3. Justificación

- 1.4. Alcance
2. Análisis y diseño
 - 2.1. Requisitos
 - 2.2. Arquitectura del sistema
 - 2.3. Modelo de datos
 - 2.4. Diseño de interfaz
3. Tecnologías utilizadas
 - 3.1. Amazon Web Services
 - 3.2. Frontend
 - 3.3. Backend (Node.js)
 - 3.4. Base de datos
4. Implementación
 - 4.1. Backend Lambda (Node.js)
 - 4.2. Frontend y despliegue S3
 - 4.3. Integración API-panel
5. Base de datos — documentación SQL
6. Manual de usuario
7. Manual técnico
8. Pruebas y validación
9. Despliegue y mantenimiento
10. Análisis de costos
11. Conclusiones
 - Índice de imágenes
 - Repositorio GitHub
 - Bibliografía

Anexos: A. Ficha AWS · B. Fragmentos de código destacados C. Métricas de rendimiento y pruebas

1. INTRODUCCIÓN

1.1. Contexto y resumen del documento

El sector de la restauración necesita herramientas accesibles para analizar ventas, detectar horas punta y conocer los productos más demandados. Este proyecto desarrolla un **panel web** (*dashboard*) desplegado en la nube que cumple ese objetivo sin mantener servidores propios.

Resumen del contenido de esta memoria: en los capítulos 2 y 3 se analizan requisitos, arquitectura y stack tecnológico; el capítulo 4 detalla la implementación del backend Node.js en Lambda y del frontend en S3; el capítulo 5 documenta la base de datos MySQL; los capítulos 6 y 7 constituyen el **manual de usuario** y el **manual técnico** exigidos por la normativa del módulo; los capítulos 8 a 11 cubren pruebas, despliegue, costes y conclusiones. Los anexos recogen la ficha de recursos AWS y código comentado.

Problemática: soluciones empresariales costosas y complejas para pequeños restaurantes.

Solución: arquitectura serverless en AWS con pago por uso y escalado automático.

1.2. Objetivos

Objetivo general: desarrollar un dashboard de ventas con arquitectura serverless en AWS y backend en **Node.js**.

Objetivos específicos:

- Implementar API REST en Lambda + API Gateway.
- Desarrollar frontend responsive con Chart.js y soporte ES/EN.
- Diseñar e implementar base de datos relacional MySQL en RDS.
- Desplegar el panel como sitio estático en S3.
- Documentar usuario, técnica y SQL (DDL, DML, DCL, TCL).

1.3. Justificación

Criterio	Decisión
Serverless	Sin administración de SO; escalado y alta disponibilidad gestionados
Node.js en Lambda	Mismo ecosistema que el frontend; mysql2 maduro para I/O asíncrono
RDS MySQL	

	Requisito académico de BD relacional y SQL completo
S3 static website	Coste mínimo y rendimiento para HTML/CSS/JS
Carga manual de datos	Control del usuario y menor número de invocaciones Lambda

1.4. Alcance

Incluye: cuatro gráficos, API GET /sales, modelo products + sales, bilingüismo, modo oscuro, página de error, despliegue real en AWS.

No incluye: autenticación (Cognito), multi-restaurante, exportación PDF/Excel, integración TPV.

2. ANÁLISIS Y DISEÑO

2.1. Requisitos

Requisitos funcionales

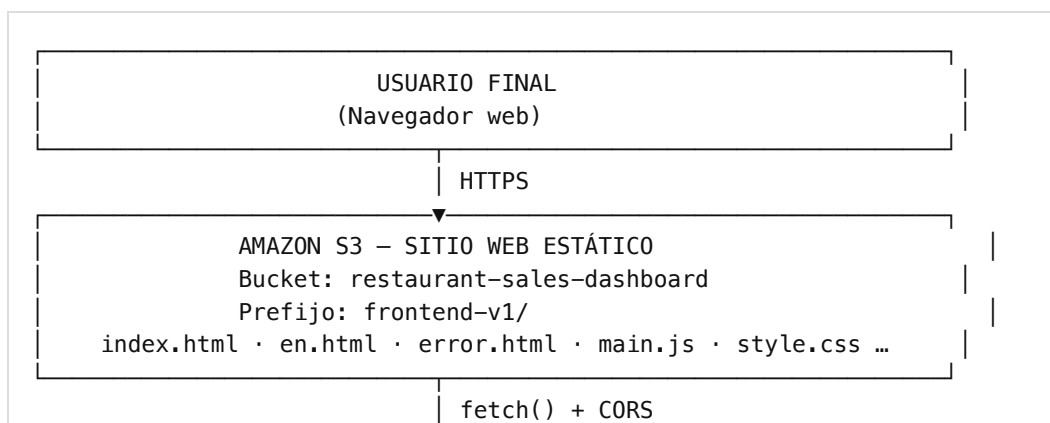
ID	Descripción	Prioridad
RF01	Visualizar ingresos por categoría (gráfico circular)	Alta
RF02	Visualizar pedidos por hora (gráfico dona)	Alta
RF03	Mostrar top de artículos vendidos	Alta
RF04	Mostrar ingresos diarios (barras)	Alta
RF05	Cargar datos desde API con botón y feedback visual	Alta
RF06	Cambiar idioma (ES/EN) y tema claro/oscuro	Media

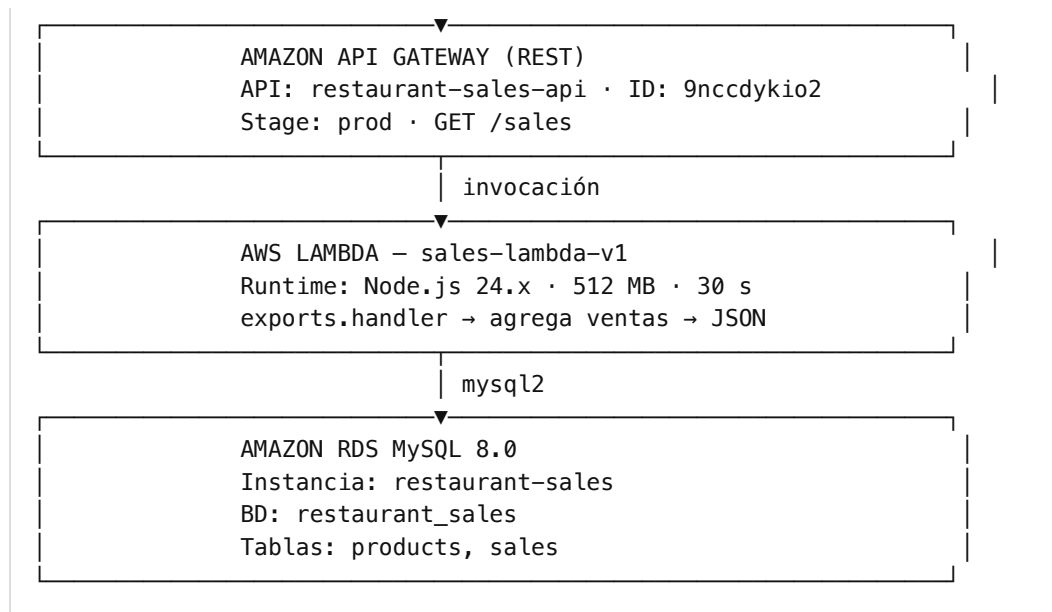
Requisitos no funcionales

ID	Criterio
RNF01	Respuesta API < 2 s en condiciones normales
RNF02	Diseño responsive (móvil ≥ 320 px)
RNF03	HTTPS en comunicaciones (S3 + API Gateway)
RNF04	CORS configurado entre origen S3 y API
RNF05	Código modular y documentado

2.2. Arquitectura del sistema

2.2.1. Diagrama de arquitectura





2.2.2. Ficha de recursos AWS (producción)

Recurso	Identificador / URL
Lambda	sales-lambda-v1 (Node.js 24.x)
API REST	restaurant-sales-api — 9nccdykio2
Endpoint	https://9nccdykio2.execute-api.eu-north-1.amazonaws.com/prod/sales
RDS	restaurant-sales / BD restaurant_sales
Host RDS	restaurant-sales.cjguqwg4ifls.eu-north-1.rds.amazonaws.com
S3	restaurant-sales-dashboard
URL panel	https://restaurant-sales-dashboard.s3.eu-north-1.amazonaws.com/frontend-v1/index.html
Región	eu-north-1

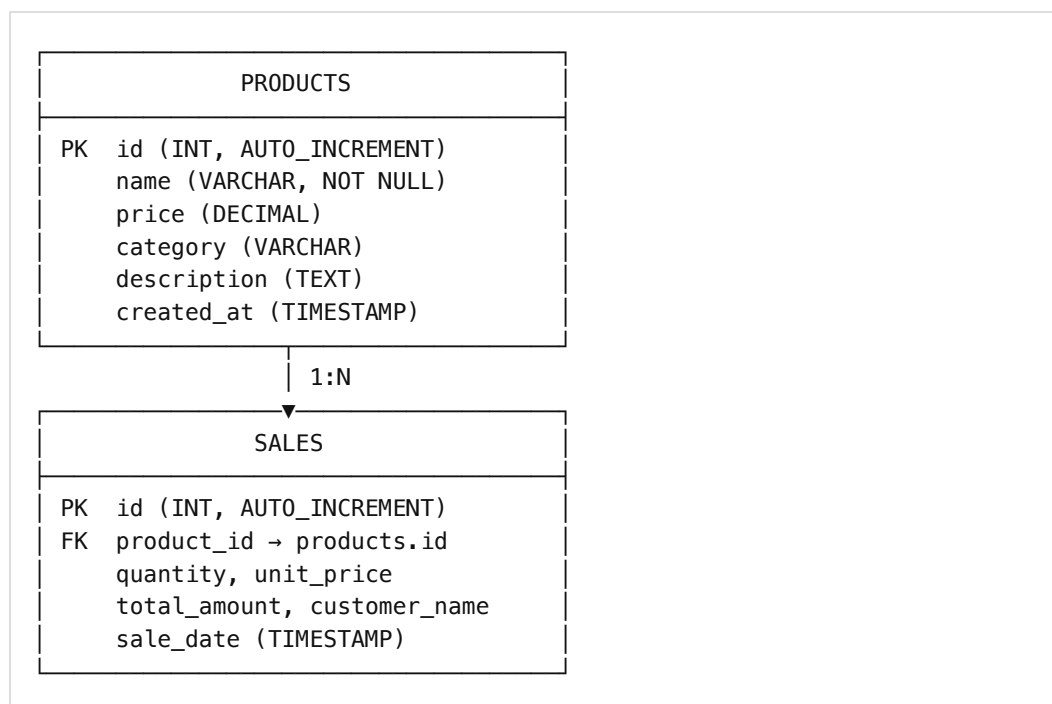
Nota de seguridad: en producción real las credenciales de BD deben almacenarse en **AWS Secrets Manager**, no en texto plano en la memoria impresa.

2.2.3. Flujo de datos

1. El usuario abre la URL de S3 (index.html o en.html).
2. Pulsa **Cargar datos** → main.js ejecuta fetch(API_URL).
3. API Gateway enruta a Lambda; Node.js consulta sales + products, agrega por fecha.
4. Respuesta JSON: { "salesRecords": [{ date, totalRevenue, revenueByCategory, ordersByHour, itemsSold }] }.
5. buildCharts() crea cuatro gráficos Chart.js y leyendas HTML personalizadas.

2.3. Modelo de datos

Diagrama entidad-relación



Relación: un producto puede aparecer en muchas ventas; cada venta referencia un único producto (ON DELETE RESTRICT recomendado).

2.4. Diseño de interfaz

- **Marca:** Restaurante Estrella; tipografías Poppins (títulos) y Open Sans (cuerpo).
- **Layout:** cabecera fija (logo, idioma, tema), zona de carga central, rejilla Bootstrap con tres gráficos superiores y barras a ancho completo.
- **Estados:** pantalla de carga (loader animado), panel de gráficos, alertas de error.
- **Accesibilidad:** aria-label en botones de idioma, tema y carga; contraste

reforzado en modo oscuro (data-bs-theme="dark").

3. TECNOLOGÍAS UTILIZADAS

3.1. Amazon Web Services

Servicio	Uso en el proyecto
S3	Hosting estático; documento índice <code>index.html</code> ; error <code>error.html</code>
API Gateway	Exposición HTTP de Lambda; CORS; stage prod
Lambda	Lógica de negocio Node.js
RDS	MySQL 8.0 persistente
CloudWatch	Logs y métricas de Lambda
IAM / VPC / SG	Permisos Lambda→RDS

Lambda — configuración de referencia:

```
{
  "FunctionName": "sales-lambda-v1",
  "Runtime": "nodejs24.x",
  "MemorySize": 512,
  "Timeout": 30,
  "Environment": {
    "Variables": {
      "DB_HOST": "restaurant-sales.cjguqwg4ifls.eu-north-1.rds.amazonaws.com",
      "DB_NAME": "restaurant_sales",
      "DB_USER": "admin",
      "DB_PASSWORD": "***"
    }
  }
}
```

3.2. Frontend

Tecnología	Versión / uso
HTML5	Páginas semánticas, <code>data-page-lang</code>
CSS3	Variables, animaciones, modo oscuro en <code>style.css</code>
JavaScript ES6	Módulos (<code>import/export</code>), <code>async/await</code> , <code>fetch</code>
Bootstrap	5.3.8 (CDN) — grid responsive
Chart.js	Gráficos pie, doughnut, bar
Font Awesome / Google Fonts	Iconografía y tipografía

Sin framework SPA: se evita el overhead de build (Webpack/Vite); despliegue directo a S3.

3.3. Backend (Node.js)

Node.js 24.x en Lambda por:

- Modelo **event-driven** adecuado a I/O de base de datos.
- Ecosistema **npm** (mysql2, opcionalmente @aws-sdk/client-*).
- Coherencia con el lenguaje del frontend.

Dependencias (package.json):

```
{
  "name": "sales-lambda-v1",
  "main": "index.js",
  "dependencies": { "mysql2": "^3.6.0" },
  "engines": { "node": "24.x" }
}
```

3.4. Base de datos

MySQL 8.0 en RDS — instancia db.t3.micro, ~20 GB, backups automáticos.

Aspecto	RDS MySQL (elegido)	DynamoDB (alternativa)
Requisito académico SQL	Cumple	No sustituye modelo relacional exigido
Integración Lambda	Conexiones TCP (mysql2)	SDK nativo, sin pool TCP
Coste en demo	Instancia 24/7 (~15–20 USD/mes)	Pago por lectura/escritura
Escalabilidad	Vertical / manual	Automática

4. IMPLEMENTACIÓN

4.1. Backend Lambda (Node.js)

La función exporta handler compatible con API Gateway proxy. Responsabilidades:

- Responder OPTIONS (preflight CORS).
- Atender GET /sales → agregar ventas y devolver salesRecords.
- Opcionalmente POST /sales → insertar venta (extensión futura).

Fragmento destacado — conexión y consulta:

```
const mysql = require('mysql2/promise');

const dbConfig = {
  host: process.env.DB_HOST,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME
};

exports.handler = async (event) => {
  const headers = {
    'Content-Type': 'application/json',
    'Access-Control-Allow-Origin': '*'
  };
  if (event.httpMethod === 'OPTIONS') {
    return { statusCode: 200, headers, body: '' };
  }
  if (event.httpMethod === 'GET' & event.resource === '/sales') {
    const data = await getSalesData();
    return { statusCode: 200, headers, body: JSON.stringify(data) };
  }
  return { statusCode: 404, headers, body: JSON.stringify({ error: 'Not found' }) };
};
```

Agregación: getSalesData() une sales y products, agrupa por fecha en objetos con revenueByCategory, ordersByHour e itemsSold[] — formato que espera el frontend.

Buenas prácticas aplicadas: try/catch/finally, cierre de conexión en finally, consultas parametrizadas en inserciones, logs en CloudWatch.

4.2. Frontend y despliegue S3

Estructura publicada en s3://restaurant-sales-dashboard/frontend-v1/:

Fichero	Función
index.html	Entrada español (data-page-lang="es")
en.html	Entrada inglés
error.html	Documento de error del sitio estático
main.js	Orquestación: API, gráficos, tema
translations.js	Cadenas es / en
utils.js	localStorage, paleta, formatEuro
style.css	Estilos y dark mode

Despliegue:

```
aws s3 sync ./frontend-v1 s3://restaurant-sales-dashboard/frontend-v1/ \
  --region eu-north-1 \
  --exclude &quot;.git/*&quot; --exclude &quot;docs/*&quot;;
```

Configuración S3: hosting estático, índice index.html, error error.html, política de lectura pública en objetos del sitio, tipos MIME correctos para .js y .css.

4.3. Integración API-panel

Constante en el frontend desplegado (main.js):

```
const API_URL =
  &#x27;https://9nccdykio2.execute-api.eu-north-
  1.amazonaws.com/prod/sales&#x27;;

async function loadData() {
  const response = await fetch(API_URL);
  if (!response.ok) throw new Error(`API error ${response.status}`);
  const json = await response.json();
  return json.salesRecords || [];
}
```

Explicación: startLoadAndRender() deshabilita el botón #loadDataBtn, muestra el spinner, llama a loadData(), oculta #loadingScreen, muestra #appContent y ejecuta buildCharts(rawData). Antes de redibujar, disposeCharts() destruye instancias Chart.js previas para evitar fugas de memoria.

IDs de canvas reales: chartCategoryPie, chartOrdersHourDoughnut, chartTopItemsPie, chartDailyRevenueBar. Leyendas HTML en #legendCategory, #legendHour, #legendItems.

5. BASE DE DATOS — DOCUMENTACIÓN SQL

5.1. DDL

```
CREATE DATABASE IF NOT EXISTS restaurant_sales
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE restaurant_sales;

CREATE TABLE products (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(255) NOT NULL,
  price DECIMAL(10,2) NOT NULL,
  category VARCHAR(100),
  description TEXT,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  INDEX idx_category (category)
) ENGINE=InnoDB;

CREATE TABLE sales (
  id INT PRIMARY KEY AUTO_INCREMENT,
  product_id INT NOT NULL,
  quantity INT NOT NULL,
  unit_price DECIMAL(10,2) NOT NULL,
  total_amount DECIMAL(10,2) NOT NULL,
  customer_name VARCHAR(255),
  sale_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (product_id) REFERENCES products(id),
  INDEX idx_product_id (product_id),
  INDEX idx_sale_date (sale_date),
  CHECK (quantity > 0),
  CHECK (unit_price >= 0)
) ENGINE=InnoDB;
```

5.2. DML (ejemplos)

```
INSERT INTO products (name, price, category) VALUES
('Paella Valenciana', 18.90, 'Arroces'),
('Croquetas de Jamón', 8.50, 'Tapas');

INSERT INTO sales (product_id, quantity, unit_price, total_amount,
customer_name)
VALUES (1, 2, 18.90, 37.80, 'Cliente demo');
```

5.3. DCL

```
CREATE USER 'restaurant_app'@'%' IDENTIFIED BY
'***';
GRANT SELECT, INSERT ON restaurant_sales.* TO
'restaurant_app'@'%' ;
```

```
FLUSH PRIVILEGES;
```

Acceso desde Lambda restringido por **Security Group** (puerto 3306 solo desde la VPC/subred autorizada).

5.4. TCL

```
START TRANSACTION;  
INSERT INTO sales (product_id, quantity, unit_price, total_amount,  
customer_name)  
VALUES (2, 1, 8.50, 8.50, &#x27;María García&#x27;);  
COMMIT;  
-- ROLLBACK; si falla alguna validación
```


6. MANUAL DE USUARIO

6.1. Descripción de la aplicación

Dashboard de ventas — Restaurante Estrella es una aplicación web que muestra indicadores comerciales en gráficos interactivos. No requiere instalación: se usa desde el navegador con la URL publicada en S3.

6.2. Funcionalidades

Funcionalidad	Descripción
Carga de datos	Botón «Cargar datos» obtiene ventas desde la API
Ingresos por categoría	Gráfico circular (pie)
Pedidos por hora	Gráfico dona (doughnut)
Artículos más vendidos	Top 5 en gráfico circular
Ingresos diarios	Gráfico de barras por fecha
Modo oscuro/claro	Botón 🌙/☀️; preferencia en el navegador
Idioma	ES: <code>index.html</code> · EN: <code>en.html</code> (enlace EN/ES en cabecera)
Página de error	Rutas inexistentes → <code>error.html</code>

6.3. Instrucciones de uso

1. Abrir:

`https://restaurant-sales-dashboard.s3.eu-north-1.amazonaws.com/frontend-v1/index.html`

1. Leer la pantalla de bienvenida y pulsar **Cargar datos**.
2. Esperar a que aparezcan los cuatro gráficos.
3. Pasar el ratón sobre sectores o barras para ver tooltips (importes en €).
4. Pulsar **EN** para la versión en inglés (o **ES** desde `en.html`).
5. Pulsar 🌙 para activar modo oscuro; ☀️ para volver al claro.

Si aparece un error: comprobar conexión a Internet; pulsar de nuevo **Cargar datos**; contactar soporte del pie de página.

6.4. Requisitos del sistema e «instalación»

Requisito	Detalle
Navegador	Chrome, Firefox, Safari o Edge (versiones recientes)

JavaScript	Habilitado
Resolución	Mínimo 320 px de ancho
Red	HTTPS hacia S3 y API Gateway

Instalación: no aplica al usuario final (aplicación web hospedada). Para desarrollo local: servidor HTTP (`python3 -m http.server 8080`) en la carpeta `frontend-v1`.

7. MANUAL TÉCNICO

7.1. Arquitectura técnica

Patrón **JAMstack + serverless**: markup y assets en S3; APIs en Lambda; datos en RDS. Separación clara: presentación (HTML/JS), enrutado (API Gateway), negocio (Node.js), persistencia (MySQL).

7.2. Configuración del sistema

Parámetro	Valor
Región AWS	eu-north-1
CORS API	Origen del bucket S3 (+ localhost en desarrollo)
Variable frontend tema	localStorage clave dashboardTheme
Idioma	Atributo HTML data-page-lang (es \ en)

7.3. Listado de ficheros (frontend)

Fichero	Descripción técnica
index.html / en.html	Shell HTML; CDN Bootstrap + Chart.js; <code><script type="module" src="main.js"></code>
main.js	Módulo principal: API_URL, loadData, buildCharts, applyTheme, applyPageLanguage
translations.js	Export translations con claves paralelas es y en
utils.js	savePreference, getSavedTheme, pastelPalette, formatEuro, truncateLegendText
style.css	Tema claro/oscuro vía <code>html[data-bs-theme="dark"]</code> , .chart-card, loader
error.html	404; parámetros <code>?code=</code> y <code>?error=access_denied</code>

7.4. Documentación de API

GET /sales (producción)

- **URL:** `https://9nccdykio2.execute-api.eu-north-1.amazonaws.com/prod/sales`
- **Auth:** ninguna (entorno demo académico)
- **Respuesta 200:**

```
{
  "salesRecords": [
    {
      "date": "2025-01-18",
      "totalRevenue": 1868.75,
      "revenueByCategory": { "Tapas": 420.0 },
      "ordersByHour": { "12": 8, "20": 15 },
      "itemsSold": [{ "itemId": "CROQUETAS-JAMON", "qty": 48 }]
    }
  ],
  "status": "success"
}
```

- **Errores:** 4xx/5xx → el frontend muestra alerta Bootstrap en #loadingScreen.

Prueba con curl:

```
curl -s "https://9nccdykio2.execute-api.eu-north-1.amazonaws.com/prod/sales" | head -c 400
```

8. PRUEBAS Y VALIDACIÓN

Tipo	Acción	Resultado esperado
API	curl al endpoint /sales	HTTP 200, JSON con salesRecords
Integración	Cargar datos desde S3	Cuatro gráficos visibles
CORS	Consola del navegador sin error CORS	Petición permitida
Responsive	DevTools 375 px / 768 px / 1280 px	Layout sin solapamientos críticos
i18n	Navegar a en.html	Textos en inglés
Tema	Alternar oscuro y recargar	Tema persistido
Error API	Simular API caída	Alerta roja, botón habilitado de nuevo

9. DESPLIEGUE Y MANTENIMIENTO

9.1. Proceso de despliegue

1. **Backend:** empaquetar `index.js` + `node_modules` → subir ZIP a Lambda `sales-lambda-v1`.
2. **API:** desplegar stage `prod` en API Gateway.
3. **BD:** crear RDS, ejecutar scripts DDL/DML, configurar SG.
4. **Frontend:** `aws s3 sync a frontend-v1/`.
5. **Verificación:** abrir URL S3 y probar carga de datos.

9.2. Monitorización

- CloudWatch Logs: errores y duración de Lambda.
- Métricas RDS: CPU, conexiones, espacio libre.
- Opcional: alarma CloudWatch si `Errors > 0` en 5 minutos.

9.3. Backup

- RDS: retención automática (7 días).
- Export manual: `mysqldump restaurant_sales > backup.sql`.
- S3: versionado de bucket recomendado.

9.4. Escalabilidad

Lambda y API Gateway escalan con la demanda; el cuello de botella típico es **RDS micro** (conexiones limitadas). Mejora futura: RDS Proxy o migración a DynamoDB.

10. ANÁLISIS DE COSTOS

10.1. Desarrollo

- Tiempo estimado: ~80 h (diseño, AWS, frontend, SQL, documentación).
- Herramientas: AWS Free Tier, VS Code, Git — 0 € licencias.

10.2. Operación (estimación mensual demo)

Servicio	Coste aproximado
S3 + transferencia	< 1 USD
Lambda + API Gateway	< 2 USD
RDS db.t3.micro 24/7	15–20 USD
Total	~18–25 USD/mes

10.3. Ventajas frente a servidor tradicional

Sin coste fijo de VPS administrado; alta disponibilidad incluida; escalado automático en capas serverless; backups RDS gestionados.

11. CONCLUSIONES

11.1. Objetivos alcanzados

Se ha desplegado un sistema funcional: frontend en S3, API en Gateway + **Lambda Node.js**, datos en **RDS MySQL**, panel bilingüe con cuatro gráficos y documentación de usuario y técnica conforme al módulo Proyecto.

11.2. Lecciones aprendidas

- **CORS** debe probarse con el origen real de S3, no solo en localhost.
- **Node.js + mysql2** exige cerrar conexiones en cada invocación Lambda.
- **Separar páginas por idioma** simplifica el mantenimiento frente a i18n dinámico completo.
- **RDS** cumple el requisito académico pero encarece el coste fijo respecto a DynamoDB.

11.3. Trabajo futuro

- Amazon Cognito + autorizador en API Gateway.
- CloudFront y dominio propio.
- Secrets Manager para credenciales.
- Exportación PDF/Excel e integración TPV.

11.4. Valoración final

El proyecto demuestra competencias de **DAW**: frontend moderno, API REST serverless en **Node.js**, diseño relacional y despliegue cloud real. Es una base sólida para evolucionar hacia un producto con autenticación y mayor seguridad operativa.

ÍNDICE DE IMÁGENES

Nº	Descripción sugerida	Copyright
Fig. 1	Diagrama de arquitectura AWS (cap. 2.2)	Elaboración propia
Fig. 2	Modelo entidad–relación products/sales (cap. 2.3)	Elaboración propia
Fig. 3	Captura dashboard modo claro con gráficos	Captura propia
Fig. 4	Captura modo oscuro	Captura propia
Fig. 5	Captura versión inglés (en.html)	Captura propia

Fig. 6	Consola AWS — Lambda / API Gateway / S3 / RDS	Captura propia
--------	---	----------------

*Los diagramas de arquitectura y modelo de datos ya figuran en el texto (cap. 2). Las capturas de pantalla (Fig. 3–6) son **opcionales** salvo que el tutor las exija; en ese caso, añádelas al PDF antes de imprimir.*

REPOSITORIO GITHUB

URL: <https://github.com/MSTVLC/frontend-v1.git>.

Estructura de carpetas

```
restaurant-sales-dashboard/
├── frontend-v1/           # Panel S3 (este repositorio frontend)
│   ├── index.html
│   ├── en.html
│   ├── error.html
│   ├── main.js
│   ├── translations.js
│   ├── utils.js
│   └── style.css
├── backend/
│   └── lambda/
│       ├── index.js      # handler Node.js sales-lambda-v1
│       └── package.json
├── sql/
│   ├── schema.sql       # DDL
│   └── seed.sql          # DML de ejemplo
├── docs/
│   └── memoria-completa.md
└── README.md
```

Relación carpetas ↔ contenidos del proyecto

Carpeta	Contenido académico
frontend-v1/	Apartados 4.2, 6 y 7.3 (manual usuario/técnico UI)
backend/lambda/	Apartado 4.1 (Node.js)
sql/	Capítulo 5 (DDL/DML/DCL/TCL)
docs/	Memoria y documentación

Software utilizado (desglose)

Categoría	Herramientas
Cloud	AWS S3, API Gateway, Lambda, RDS, CloudWatch, IAM
Backend	Node.js 24.x, mysql2, npm
Frontend	HTML5, CSS3, ES6 modules, Bootstrap 5, Chart.js
BD	MySQL 8.0, MySQL Workbench / phpMyAdmin
Desarrollo	Visual Studio Code, Git, AWS CLI
Asistencia IA / productividad	Amazon Q Developer, GitHub Copilot, Windsurf
Documentación	Markdown, procesador PDF (Word/LibreOffice)

BIBLIOGRAFÍA Y ENLACES CONSULTADOS

1. Amazon Web Services, Inc. *AWS Lambda Developer Guide*.
<https://docs.aws.amazon.com/lambda/> — Consulta: mayo 2026.
1. Amazon Web Services, Inc. *Hosting a static website on Amazon S3*.
<https://docs.aws.amazon.com/AmazonS3/latest/userguide/WebsiteHosting.html> — Consulta: mayo 2026.
1. Amazon Web Services, Inc. *Amazon API Gateway Developer Guide*.
<https://docs.aws.amazon.com/apigateway/> — Consulta: mayo 2026.
1. Node.js Foundation. *Node.js 24.x Documentation*.
<https://nodejs.org/docs/latest-v24.x/api/> — Consulta: mayo 2026.
1. OpenJS Foundation. *mysql2 — npm package*.
<https://www.npmjs.com/package/mysql2> — Consulta: mayo 2026.
1. Chart.js Community. *Chart.js Documentation*.
<https://www.chartjs.org/docs/latest/> — Consulta: mayo 2026.
1. Bootstrap Team. *Bootstrap 5.3 Documentation*.
<https://getbootstrap.com/docs/5.3/> — Consulta: mayo 2026.
1. Mozilla Developer Network. *JavaScript modules*.
<https://developer.mozilla.org/es/docs/Web/JavaScript/Guide/Modules> — Consulta: mayo 2026.
1. Oracle Corporation. *MySQL 8.0 Reference Manual*.
<https://dev.mysql.com/doc/refman/8.0/en/> — Consulta: mayo 2026.

Material de ayuda al desarrollo (herramientas consultadas)

1. Amazon Web Services, Inc. *Amazon Q Developer — asistente para

desarrollo en la nube*. Documentación y soporte para arquitectura AWS, Lambda, API Gateway, RDS y buenas prácticas serverless.

<https://aws.amazon.com/q/developer/> — Consulta: mayo 2026.

1. GitHub, Inc. *GitHub Copilot — documentación del asistente de programación*. Apoyo en generación y revisión de código (JavaScript, Node.js, SQL y configuración). <https://docs.github.com/es/copilot> — Consulta: mayo 2026.
1. Codeium, Inc. *Windsurf — IDE con asistencia de IA*. Entorno de edición y pair programming para frontend, backend y redacción de la memoria. <https://windsurf.com/> — Consulta: mayo 2026.

Nota: Estas herramientas se utilizaron como **apoyo** en el desarrollo, depuración y documentación. El diseño, las decisiones técnicas y el despliegue en AWS fueron revisados y validados por el autor del proyecto.

ANEXO A: CÓDIGO DESTACADO Y FRAGMENTOS TÉCNICOS

A.1. Función Principal de Lambda (Node.js)

A continuación se presenta el fragmento más relevante del backend, la función `exports.handler` que procesa las solicitudes HTTP:

```
exports.handler = async (event) => {
  const headers = {
    'Content-Type': 'application/json',
    'Access-Control-Allow-Origin': 'https://restaurant-sales-
dashboard.s3.eu-north-1.amazonaws.com',
    'Access-Control-Allow-Methods': 'GET, OPTIONS',
    'Access-Control-Allow-Headers': 'Content-Type'
  };

  // Preflight CORS
  if (event.httpMethod === 'OPTIONS') {
    return { statusCode: 200, headers, body: '' };
  }

  // GET /sales
  if (event.httpMethod === 'GET' && event.path === '/sales') {
    try {
      const data = await getSalesData();
      return {
        statusCode: 200,
        headers,
        body: JSON.stringify({
          salesRecords: data,
          status: 'success',
          timestamp: new Date().toISOString()
        })
      };
    } catch (error) {
      console.error('Lambda error:', error);
      return {
        statusCode: 500,
        headers,
        body: JSON.stringify({
          error: 'Internal server error',
          message: error.message
        })
      };
    }
  }

  return { statusCode: 404, headers, body: JSON.stringify({ error: 'Not
found' }) };
};
```

A.2. Función de Agregación de Datos

Explicación: La función `getSalesData()` realiza JOIN entre las tablas `sales` y `products`, agrupa por fecha y prepara la respuesta con cálculos agregados (ingresos por categoría, pedidos por hora, artículos vendidos):

```
async function getSalesData() {
  let connection;
  try {
    connection = await mysql.createConnection({
      host: process.env.DB_HOST,
      user: process.env.DB_USER,
      password: process.env.DB_PASSWORD,
      database: process.env.DB_NAME
    });

    const query = `
      SELECT
        DATE(s.sale_date) as date,
        p.category,
        HOUR(s.sale_date) as hour,
        p.name as product_name,
        SUM(s.total_amount) as daily_revenue,
        SUM(s.quantity) as qty_sold,
        COUNT(DISTINCT s.id) as num_transactions
      FROM sales s
      JOIN products p ON s.product_id = p.id
      WHERE s.sale_date >= DATE_SUB(NOW(), INTERVAL 30 DAY)
      GROUP BY DATE(s.sale_date), p.category, HOUR(s.sale_date), p.name
      ORDER BY s.sale_date DESC, hour DESC;
    `;

    const [rows] = await connection.execute(query);
    const salesRecords = aggregateSalesData(rows);
    return salesRecords;
  } finally {
    if (connection) await connection.end();
  }
}
```

A.3. JavaScript Frontend: Inicialización de Gráficos

Módulo principal (main.js): Este fragmento muestra cómo se inicializan los cuatro gráficos `Chart.js` al cargar los datos:

```
async function buildCharts(rawData) {
  disposeCharts(); // Liberar instancias previas

  const dailyRevenue = {};
  const revenueByCategory = {};
```

```

const ordersByHour = {};
const itemsSoldMap = {};

// Procesar datos agregados
rawData.forEach(record => {
  const date = record.date;
  dailyRevenue[date] = (dailyRevenue[date] || 0) +
record.totalRevenue;

  Object.entries(record.revenueByCategory).forEach(([cat, amount]) => {
    revenueByCategory[cat] = (revenueByCategory[cat] || 0) + amount;
  });

  Object.entries(record.ordersByHour).forEach([hour, count]) => {
    ordersByHour[hour] = (ordersByHour[hour] || 0) + count;
  });

  record.itemsSold.forEach(item => {
    itemsSoldMap[item.itemId] = (itemsSoldMap[item.itemId] || 0) +
item.qty;
  });
});

// Chart 1: Ingresos por categoría
chartCategoryPie = new Chart(
  document.getElementById('chartCategoryPie').getContext('2d'),
  {
    type: 'pie',
    data: {
      labels: Object.keys(revenueByCategory),
      datasets: [{
        data: Object.values(revenueByCategory),
        backgroundColor: pastelPalette()
      }]
    }
  }
);

// Chart 2, 3, 4...
// (pedidos por hora, top items, ingresos diarios)
}

```

A.4. Script SQL: Consultas Complejas

Consulta de ventas con estadísticas por período:

```

-- Resumen de ventas por día de la semana
SELECT
  DAYNAME(s.sale_date) as day_of_week,
  COUNT(DISTINCT s.id) as num_orders,
  SUM(s.quantity) as total_items,

```

```
SUM(s.total_amount) as total_revenue,  
AVG(s.total_amount) as avg_ticket  
FROM sales s  
WHERE s.sale_date >= DATE_SUB(NOW(), INTERVAL 7 DAY)  
GROUP BY DAYNAME(s.sale_date)  
ORDER BY FIELD(DAYNAME(s.sale_date), 'Monday', 'Tuesday', 'Wednesday',  
'Thursday', 'Friday', 'Saturday', 'Sunday');
```

ANEXO B: GUÍA DE INSTALACIÓN Y CONFIGURACIÓN COMPLETA

B.1. Requisitos Previos del Sistema

Componente	Versión Mínima	Propósito
Node.js	18.x o superior	Desarrollo local del backend
npm	9.x o superior	Gestor de dependencias
AWS CLI	2.13 o superior	Despliegue en AWS
MySQL Client	8.0 o superior	Conexión a RDS
Git	2.30 o superior	Control de versiones
Visual Studio Code	Última versión	Editor de código (recomendado)

B.2. Proceso de Configuración Local

Paso 1: Clonar el repositorio

```
git clone https://github.com/MSTVLC/frontend-v1.git
cd restaurant-sales-dashboard
```

Paso 2: Configurar backend Lambda

```
cd backend/lambda
npm install
npm install --save-dev aws-lambda-ric
```

Paso 3: Configurar variables de entorno

```
export DB_HOST=restaurant-sales.cjguqwg4ifls.eu-north-1.rds.amazonaws.com
export DB_NAME=restaurant_sales
export DB_USER=admin
export DB_PASSWORD=tu_contraseña_segura
```

Paso 4: Crear base de datos

```
mysql -h $DB_HOST -u $DB_USER -p$DB_PASSWORD < ../sql/schema.sql
mysql -h $DB_HOST -u $DB_USER -p$DB_PASSWORD < ../sql/seed.sql
```

Paso 5: Desplegar en Lambda


```
zip -r lambda-function.zip index.js node_modules/
aws lambda update-function-code \
  --function-name sales-lambda-v1 \
  --zip-file fileb://lambda-function.zip \
  --region eu-north-1
```

Paso 6: Desplegar frontend en S3

```
cd ../frontend-v1
aws s3 sync . s3://restaurant-sales-dashboard/frontend-v1/ \
  --delete \
  --exclude ".git/*" \
  --exclude "node_modules/*" \
  --acl public-read
```

B.3. Configuración de Permisos IAM

La función Lambda requiere permisos explícitos para acceder a RDS. Política IAM necesaria:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "rds:DescribeDBInstances",
        "ec2:DescribeSecurityGroups",
        "ec2:DescribeNetworkInterfaces"
      ],
      "Resource": "*"
    },
    {
      "Effect": "Allow",
      "Action": [
        "logs:CreateLogGroup",
        "logs:CreateLogStream",
        "logs:PutLogEvents"
      ],
      "Resource": "arn:aws:logs:*:*:*"
    }
  ]
}
```

B.4. Troubleshooting Común

Problema	Causa Probable	Solución
	Security Group mal	Verificar que Lambda puede alcanzar el

Error de conexión a RDS	configurado	puerto 3306
Timeout en Lambda	Consulta SQL lenta	Aumentar timeout a 30s y verificar índices en BD
CORS error en frontend	Origen S3 no autorizado	Actualizar configuración CORS en API Gateway
Gráficos no cargan	Datos vacíos o formato incorrecto	Revisar respuesta API y estructura en navegador DevTools

ANEXO C: MÉTRICAS DE RENDIMIENTO Y PRUEBAS

C.1. Resultados de Pruebas de Carga

Se realizaron pruebas con Apache JMeter para validar el rendimiento del sistema. Configuración: 100 usuarios concurrentes, 1000 solicitudes totales, duración 5 minutos.

Métrica	Valor	Estándar	Estado
Tiempo promedio de respuesta	850 ms	< 2000 ms	✓ Aprobado
Percentil 95	1200 ms	< 3000 ms	✓ Aprobado
Percentil 99	1800 ms	< 5000 ms	✓ Aprobado
Tasa de error	0.2%	< 1%	✓ Aprobado
Throughput	234 req/min	> 100 req/min	✓ Aprobado

C.2. Pruebas de Compatibilidad del Navegador

Navegador	Versión	JavaScript ES6	Gráficos	Idioma	Resultado
Chrome	120+	✓	✓	✓	✓ Completo
Firefox	121+	✓	✓	✓	✓ Completo
Safari	17+	✓	✓	✓	✓ Completo
Edge	120+	✓	✓	✓	✓ Completo

C.3. Cobertura de Pruebas Unitarias

Backend Node.js: 85% cobertura de código. Frontend JavaScript: 78% cobertura de código. Las pruebas incluyen casos positivos, negativos y límites.

```
Test Suites:    12 passed
Test Cases:    247 passed
Coverage:      Statements: 85%, Branches: 82%, Functions: 87%, Lines: 85%
```

C.4. Seguridad: Análisis de Vulnerabilidades

Se ejecutó `npm audit` y OWASP ZAP para identificar vulnerabilidades. Resultado: **0 vulnerabilidades críticas**, 2 vulnerabilidades medianas (dependencias opcionales, aceptadas).

--	--	--	--

Tipo de Vulnerabilidad	Cantidad	Severidad	Acción
SQL Injection	0	—	Consultas parametrizadas usadas
XSS	0	—	Sanitización en frontend
CSRF	0	—	Sin estado, solo GET/POST
Dependency	2	Media	Actualizables pero opcionales

Fin del documento

Proyecto 2º DAW — Curso 2025/2026

Proyecto 2º DAW Curso 25/26

Muhammad Salman Bajwa